

QUEUE

Q4)Queue Implementation using Linked List

DS	Session	Queue	Question Information	Level 1	Challenge 54
----	---------	-------	----------------------	---------	--------------

Question description

Joe root is a Placement trainer, he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept. Joe root usually gives interesting problems to the students, to make them love the DS.

One such day Joe root provided to the final year students to solve the task such that, Queue implementation with arrays as using linked list for implementing queue and delete an element from the queue using linked list concept.

Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

enqueue(data)

- Build a new node with given data.
- Check if the queue is empty or not.
- If queue is empty then assign new node to front and rear.
- Else make next of rear as new node and rear as new node.

dequeue()

- Check if queue is empty or not.
- If queue is empty then dequeue is not possible.
- Else store front in temp
- And make next of front as front.

print()

- Check if there is some data in the queue or not.
- If the queue is empty print "No data in the queue."
- Else define a node pointer and initialize it with front.
- Print data of node pointer until the next of node pointer becomes NULL.

Constraints:

0<size<100
0<data<1000

Input format:

First line indicates the size of the queue
Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept
second line indicates the one element dequeued from the queue using linked list concept.

1 / 2

🔍 ↺ 🔍

CODE:

```

C main.c > main
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *next;
7 };
8
9 struct node *front = NULL;
10 struct node *rear = NULL;
11
12 void enqueue(int d) {
13     struct node* new_n;
14     new_n = (struct node*)malloc(sizeof(struct node));
15     new_n->data = d;
16     new_n->next = NULL;
17     if (front == NULL) {
18         front = rear = new_n;
19     } else {
20         rear->next = new_n;
21         rear = new_n;
22     }
23 }
24
25 void dequeue() {
26     if (front != NULL) {
27         struct node *temp = front;
28         front = front->next;
29         if (front == NULL) rear = NULL;
30         free(temp);
31     }
32 }
33
34 void print() {
35     struct node *temp = front;
36     while (temp != NULL) {
37         printf(format: "%d", temp->data, temp->next ? " " : "");
38         temp = temp->next;
39     }
40     printf(format: "\n");
41 }
42
43 int main(void) {
44     int n, val;
45     if (scanf(format: "%d", &n) != 1) return 0;
46
47     for (int i = 0; i < n; i++) {
48         scanf(format: "%d", &val);
49         enqueue(d: val);
50     }
51
52     print();
53     dequeue();
54     print();
55
56     return 0;
57 }

```

OUTPUT:

```

[suraj@suraj-83gs test]$ ./main
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
[suraj@suraj-83gs test]$

```